Lab 34: Unknown requirements? Let Claude interview you first

Topic 3: Claude Code Configuration & Workflows · Task 3.5: Apply iterative refinement techniques for progressive improvement

Runs on Simulator 5: Claude Code for Continuous Integration · Self-practice

Source: SkillCertPro practice exam Part 1, Q14 (unofficial third-party question)

The question

This team is designing a new automated reviewer for flaky-test risk. The behavior has unresolved product and workflow questions, including when comments are actionable, when they are noise, and what verification should prove. What should the team do before asking Claude Code to implement it?

A. Run an interactive planning session where Claude uses AskUserQuestion , writes a self-contained SPEC.md , then implement it from a fresh session.

B. Run a delegated subagent via the Agent tool to ask AskUserQuestion prompts, then pass its transcript to the implementer.

C. Run direct execution with a detailed review checklist in the prompt, then adjust the workflow after the first CI failures.

D. Run a scheduled autonomous Claude Code job that infers missing requirements from recent pull request discussions.

Simulate it live

```
# In the browser: open Simulator 5, then Lab 34
cd labs
uv run serve.py

# In Claude Code, from the labs folder
/lab 34

# In a terminal, from simulators/simulator-5-ci-cd/
uv run casekit.py --case UC-14
```

What you should see

spec.md holds open questions, interfaces, out-of-scope items and verification.

Claude writes spec.md with the open questions it would have asked through AskUserQuestion, the files and interfaces, what is out of scope and the verification step. Implementation starts from a fresh session with that file.

Open `simulators/simulator-5-ci-cd/evidence/` and find it yourself.

The answer

A. Interview, write a self-contained SPEC.md, implement in a fresh session.

For larger or ambiguous work, let Claude interview the developer first. Start from a minimal feature prompt and have Claude use AskUserQuestion (a built-in tool for multiple-choice clarifying questions) to explore implementation, developer experience, edge cases, concerns and trade-offs. Then have it write a self-contained SPEC.md that names the relevant files and interfaces, states what is out of scope, and includes an end-to-end verification step. Implementing from a fresh session then works against concrete acceptance criteria. Fuzzy criteria are a main source of false positives in CI review.

Why the others fail

B. According to the video, AskUserQuestion is not currently available to subagents delegated through the Agent tool. A raw transcript is also a poor handoff compared with a spec.

C. With the criteria unresolved, this ships the false positives first and fixes them after CI failures, the expensive order.

D. Scheduled autonomous work cannot ask clarifying questions, and inferring requirements from PR chatter guesses at product decisions.

Check yourself

- spec.md was written
- it holds the four sections the interview pattern produces
- learner names AskUserQuestion as the interactive tool and a fresh session as the implementation start

Now change one thing

Fix the loop in repo/stock/totals.py and the caller in repo/stock/jobs.py, rerun, and watch the blocking count fall.

Third-party practice material from SkillCertPro, reconstructed from a video transcript in the Tertiary Markdown Vault. Not a real exam item. Wording may differ from the original.